Tema 2: TIPOS ABSTRACTOS DE DATOS

1. Abstraccción de Datos y Tipos Abstractos de Datos (TAD).
2. Especificación Algebraica de un TAD.
   2.1. Introducción
   2.2. Signatura de una especificación algebraica
   2.3. Ecuaciones de una especificación algebraica
3. Construcción de especificaciones.
   3.1. Operaciones: clasificación
   3.2. Escritura de ecuaciones
   3.3. Operaciones parciales
   3.4. Especificaciones genéricas
4. Paso de la especificación a la implementación de un TAD.

1.- Abstracciones de datos y Tipos Abstractos de Datos.

- **Abstracción de Datos:** manipulación de los datos a nivel lógico y abstracción de su implementación física en una determinada máquina o en un determinado lenguaje de programación.
- Tipos básicos o estándar: entero, carácter, booleano, etc.
- Tipos simples definidos por el programador: enumerado, subrango
- Tipos estructurados: array, registro → constructores genéricos (**genericidad**)
- Los TAD constituyen una forma de generalizar y encapsular los aspectos más importantes de la información que manejamos dentro del programa, olvidando hasta el momento de la implementación como se va a representar esa información dentro del ordenador.
- El TAD se puede ver como un conjunto de valores y operaciones que se definen sobre ellos mediante una especificación independiente de cualquier implementación

- Los TAD definen un nuevo modelo de datos independiente de su representación real en memoria, con sus operaciones relacionadas definidas, también, de forma abstracta. Esto permite que pueda ser reutilizado luego en otros programas.

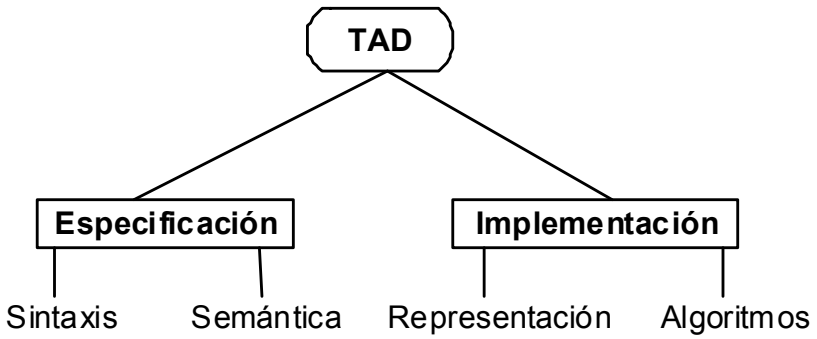
- La definición de TAD puede dividirse en 2 niveles:
    1. Una definición o especificación del TAD. Consiste en Definir el TAD y sus operaciones, de forma que cumple que es.
        - Visible al usuario
        - Precisa, legible y no ambigua

- Recoge el Nombre del tipo + sintaxis y semántica de las operaciones

2. Una implementación. Se profundiza en la definición del TAD y en la implementación de sus operaciones.
   - Oculta al usuario
   - Estructurada, eficiente y legible
   - Elección de la representación más adecuada + algoritmos de las operaciones

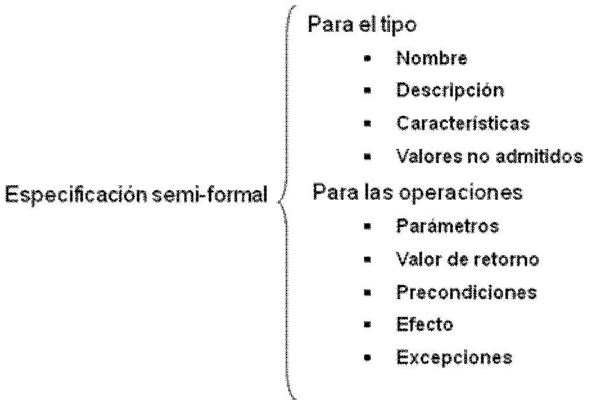
- Los usuarios del TAD sólo conocerán la definición de éste y de las operaciones definidas sobre él, sin tener acceso a su implementación.

El objetivo del TAD es separar el uso del tipo de dato de su implementación



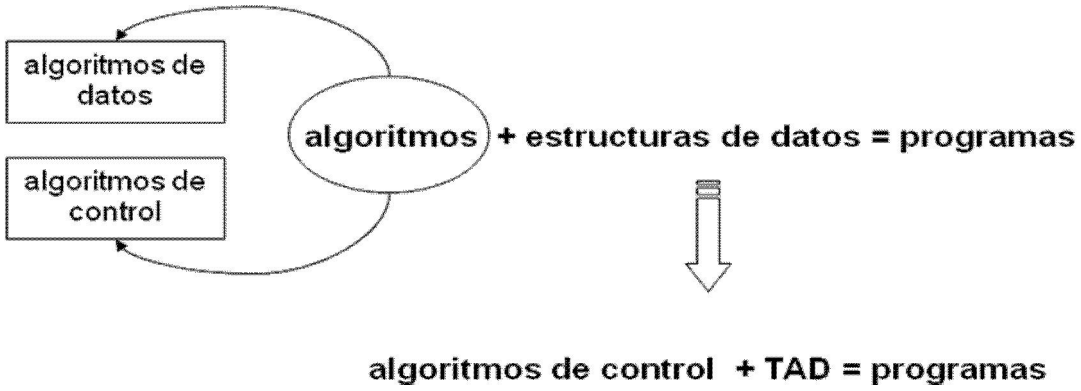
- ❑ Métodos para especificar un tipo abstracto de datos
  - ☑ Especificación formal (especificación algebraica)
  - ☑ Especificación informal (lenguaje natural)
  - ☑ Especificación semi-formal (mezcla de las anteriores)

En cursos anteriores, a la hora de especificar el TAD se hacía uso de la especificación semi-formal:



En esta asignatura vamos a trabajar con la especificación algebraica, que empezaremos a estudiar en la pregunta siguiente.

- ❑ Un lenguaje de programación soporta la implementación de tipos abstractos de datos si incluye mecanismos que permitan separar la especificación de la implementación, es decir, permita la **encapsulación** u **ocultación de la información**, que consiste en:
  - ☑ Privacidad de la representación interna. El usuario no conoce los detalles de cómo se implementan las estructuras de datos y las operaciones
  - ☑ Protección del tipo. El usuario sólo puede usar las operaciones definidas en la especificación
- ❑ La **clase** en los lenguajes OO es el mecanismo apropiado para implementar un TAD



2.- Especificación Algebraica de un TAD

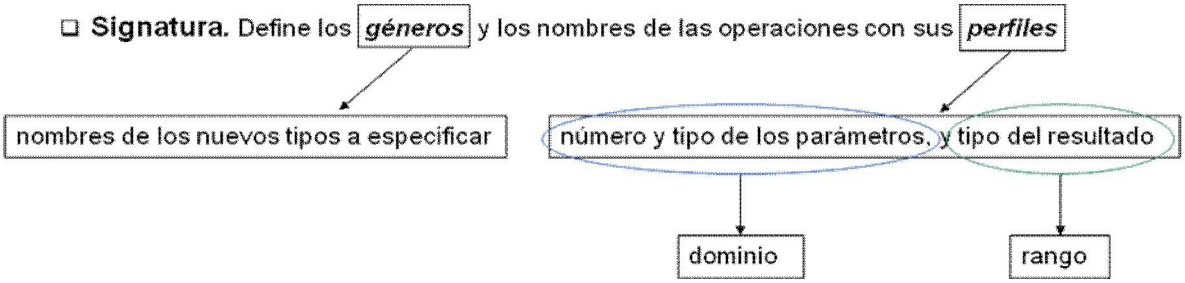
2.1 INTRODUCCIÓN

- Especificación algebraica. Técnica formal para especificar tipos abstractos de datos

VENTAJAS

- Define tipos independientemente de cualquier representación e implementación
- Consigue unanimidad en la interpretación del tipo
- Posibilita la obtención de código automáticamente a partir de la especificación algebraica.

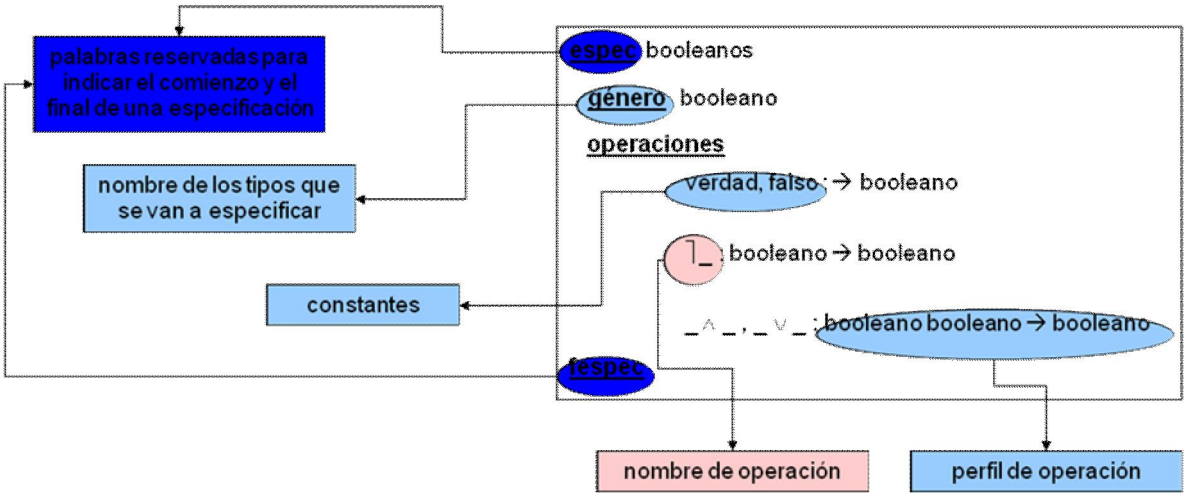
2.2 Signatura de una especificación algebraica



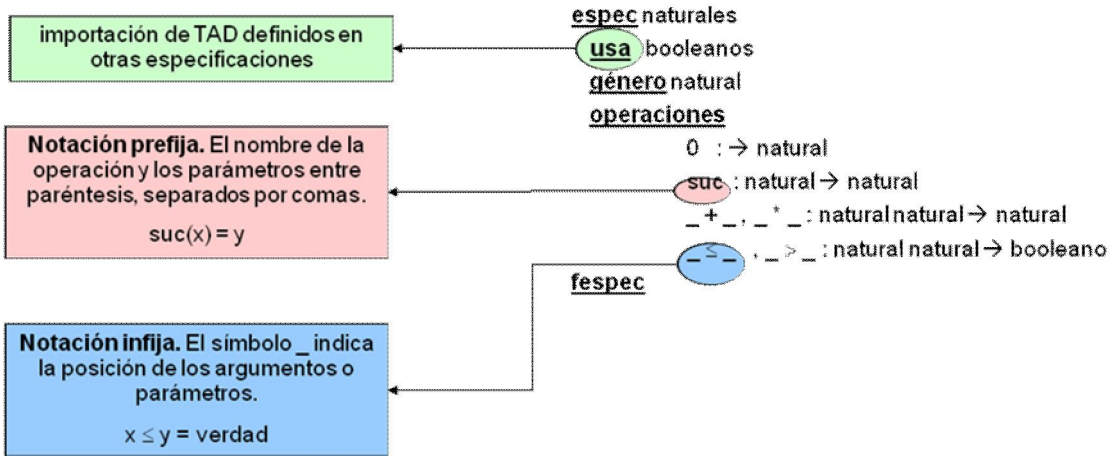
CARACTERISTICAS

- Notación funcional
- Cada operación es una función con 0 o más parámetros
- Todas las operaciones devuelven un **único** valor de un tipo determinado

Ejemplo: Signatura del TAD booleano



Ejemplo. Posible signatura de los *naturales con el 0*



- Para cada género o tipo existe un conjunto de **términos bien formados** $\Rightarrow$ sintácticamente correctos
- Cada constante es un término y mediante la aplicación sucesiva y correcta de símbolos de operaciones de una signatura se pueden construir términos
- Ejemplos de términos bien formados :

  0

  $(suc\ (suc\ (0)) * suc(0)) \leq suc(0)$

  $((verdad \wedge falso) \wedge (\neg falso)) \vee (0 > (suc\ (0)))$
- En determinadas especificaciones, un valor del tipo puede venir expresado por más de un término.

2.3 Ecuaciones de una especificación algebraica

- También se conocen con el nombre de *axiomas*
- Determinan las propiedades y el comportamiento de las operaciones
- Toda especificación debe cumplir estas dos propiedades:
  - ☑ sólo pertenecen al tipo los valores que puedan ser creados mediante términos sintácticamente correctos
  - ☑ cada término bien formado denota un único valor del tipo especificado
- Mediante las ecuaciones ha de ser posible deducir todas las equivalencias que son válidas entre los términos, es decir, identificar las secuencias de operaciones (términos) que producen el mismo resultado
  - ☑ en general, para las *operaciones binarias conmutativas* no se añadirán en la especificación las *ecuaciones de conmutatividad* de la forma: x_1 OP x_2 = x_2 OP x_1, salvo en el caso que sirvan para definir el comportamiento de dicha operación.

Ejemplo. Queremos especificar el tipo *naturales con el 0*

```
espec naturales_1
  género natural
  operaciones
      0 : → natural
      suc : natural → natural
fespec
```

- únicos valores que pueden construirse:
  0, suc (0), suc (suc (0)), suc (suc (suc (0))), etc.
- cada término denota un valor diferente

Añadimos a la especificación la operación suma

```
espec naturales_2
  género natural
  operaciones
      0 : → natural
      suc : natural → natural
      _ + _ : natural natural → natural
fespec
```

- incumple la segunda propiedad
- *"suc (0)"* y *"0 + suc (0)"* generan el natural 1

- Existen 2 puntos de vista respecto al papel de las ecuaciones en las especificaciones algebraicas:
  - ☑ igualar términos que generan el mismo valor (punto de vista algebraico)
  - ☑ definir el comportamiento de las operaciones con todas las posibles combinaciones de valores (patrones) que pueden tomar sus parámetros (punto de vista semántico)
- Formato de una ecuación: **término_1 = término_2**
  - ☑ donde *término_1* y *término_2* son términos bien formados de un mismo género

```
espec naturales_3
  género natural
  operaciones
      0 : → natural
      suc : natural → natural
      _ + _ : natural natural → natural
  ecuaciones x, y: natural
      x + 0 = x
      x + suc (y) = suc (x+y)
fespec
```

- Punto de vista algebraico
  - igualar términos que generan el mismo valor
- Punto de vista semántico
  - definir el comportamiento de las operaciones con todos los patrones o representantes del tipo

Ejemplo. Completar la especificación algebraica de los booleanos

```
espec booleanos
    género booleano
    operaciones
        verdad, falso: → booleano
        ¬_: → booleano
        _∧_ , _∨_ : booleano booleano → booleano
    ecuaciones  b: booleano
        ¬verdad = falso
        ¬falso = verdad

        b ∨ verdad = verdad
        b ∨ falso = b
        b ∧ verdad = b
        b ∧ falso =  falso
fespec
```

3.- Construcción de especificaciones

3.1. Operaciones: clasificación

- **g:** símbolo o identificador del género correspondiente al tipo que se desea especificar
- **OP(g):** conjunto de operaciones relacionadas con *g*
- Clasificación:
  - ☑ **Constructoras.** Cons(g). Conjunto de operaciones que devuelven un valor de género *g*
    - **Generadoras.** Gen(g). Subconjunto mínimo de operaciones constructoras que permite generar todos los valores del TAD que queremos especificar
    - **Modificadoras.** Mod(g). Las constructoras que no forman parte del conjunto de generadoras
  - ☑ **Observadoras**. Obs(g). Conjunto de operaciones que devuelven un valor de género diferente de *g*

Natural

```
espec naturales
  usa booleanos
  género natural
  operaciones
      0 : → natural
      suc: natural → natural
      _ + _ : natural natural → natural
      _ ≤ _ : natural natural → booleano
fespec
```

Secuencia

```
espec secuenciaDeNaturales
  usa naturales_3
  género secN
  operaciones
      [] : → secN                          {secuencia vacía}
      [_] : natural → secN                 {secuencia unitaria}
      _ ++ _ : secN secN → secN            {concatenar secuencias}
      _ ⊕ _ : natural secN → secN          {añadir un natural a una secuencia}
  ecuaciones x: natural; s, s1, s2, s3: secN
      ....
fespec
```

¿Clasificación de las operaciones?

- ❑ El conjunto Gen(g) puede ser:
  - ☑ Conjunto libre de generadoras: todo término formado sólo por constructoras generadoras denota **un valor diferente** en el tipo de datos correspondiente a *g*
  - ☑ Conjunto no libre de generadoras: dos o más términos distintos formados sólo por constructoras generadoras denotan **un mismo valor** del tipo

¿Son libres los siguientes conjuntos de generadoras?

a. Gen (natural) = {0, suc}
b. Gen (secN) = { [ ], _ ⊕ _ }
c. Gen (secN) = { [ ], [_], _ ++ _ }

3.2. Escritura de ecuaciones

- ❑ **Paso 1.** Elegir conjunto de operaciones generadoras
- ❑ **Paso 2.** Si Gen(g) es un conjunto no libre, escribir *ecuaciones entre generadoras*. Si es libre, no hacer nada
- ❑ **Paso 3.** Escribir todas las ecuaciones necesarias para definir el comportamiento de cada operación modificadora
- ❑ **Paso 4.** Escribir todas las ecuaciones necesarias para definir el comportamiento de cada operación observadora

3.3. Operaciones parciales

Ejemplo. Queremos añadir la operación *predecesor* a los naturales

```
espec naturales_4
  género natural
  operaciones
      0 : → natural
      suc : natural → natural
      parcial pred: natural → natural
dominios de definición x: natural
      pred (suc (x))
ecuaciones  x: natural
      pred (suc (x)) = x
fespec
```

- la operación predecesor no está definida para el natural 0 **(situación de error)**
- es una **operación parcial** → no puede aplicarse a todos los valores del dominio
- en la sección "**dominios de definición**" se especifican los dominios donde están definidas las operaciones parciales de la especificación

3.4. Especificaciones genéricas

- Generalmente, los TAD que representan colecciones de elementos suelen definirse de forma genérica, dejando como parámetro el elemento almacenado en la colección
- Las operaciones de estos TAD se comportan de la misma forma, independientemente del tipo de los elementos que almacena

```
espec secuenciaGenérica
  parámetro formal
      género elemento
  fpf
  género sec
  operaciones
      [] : → sec                          {secuencia vacía}
      [ _ ] : elemento → sec              {secuencia unitaria}
      _ ++ _ : sec sec → sec              {concatenar secuencias}
      _ ⊕ _ : elemento sec → sec          {añadir un elemento a una secuencia}
  ecuaciones  x: elemento; s, s1, s2, s3: sec
      s ++ [ ] = s
      [ ] ++ s = s
      (s1 ++ s2) ++ s3 = s1 ++ (s2 ++ s3)
      x ⊕ s = [x] ++ s
fespec
```

Ejemplos de Especificaciones

Ejemplo 1. Completar la especificación algebraica del TAD *Conjuntos*

```
espec conjuntos
  usa booleanos, naturales
  parámetro formal
      género elemento
  fpf
  genero conjunto
  operaciones
          ∅: → conjunto
          poner, quitar: elemento conjunto → conjunto
          _ ∪ _ , _ ∩ _ : conjunto conjunto → conjunto
          _ ∈ _ : elemento conjunto → booleano
          esVacío: conjunto → booleano
          cardinal: conjunto → natural
```

Ejemplo 2. Completar la especificación algebraica del TAD *string_reducido*

```
espec string_reducido
    usa caracteres, naturales
    género string
    operaciones
      stringVacío: → string                    {crea un string vacío}
      añade: string carácter → string          {añade un elemento al final del string}
      longitud: string → natural               {devuelve el número de caracteres que forman el string}
      parcial carac: natural string → carácter {devuelve el carácter que se encuentra en la posición indicada por el parámetro.
                                                El carácter situado a la izquierda se encuentra en la posición 1}
      concatena: string string → string        {devuelve el resultado de poner el segundo string al final del primero}
      inversa: string → string                 {devuelve el string formado por los mismos caracteres pero en orden inverso}
```

4.- Paso de la especificación a la implementación de TAD

- Dada la especificación de un TAD, su implementación consiste en:
  - ☑ Elegir un dominio concreto para cada género incluido en el TAD
    - Los dominios concretos se implementan mediante declaraciones de tipos, usando otros tipos ya implementados (por nosotros o incluidos en el propio lenguaje o en alguna librería)
  - ☑ Definir un método concreto para cada operación de la especificación
    - Los métodos concretos pueden implementarse como procedimientos o funciones aunque en la especificación algebraica sólo se admitan funciones
- La implementación debe cumplir 2 condiciones:
  - ☑ Corrección ⇒ satisfacer las ecuaciones de la especificación
    - No vamos a considerar la verificación de las implementaciones con respecto a la especificación algebraica. Requiere un desarrollo formal muy amplio. A pesar de ello, pueden establecerse relaciones "intuitivas" entre especificación e implementación
  - ☑ Privacidad y protección
    - La estructura interna del tipo y la implementación de sus operaciones deben permanecer ocultas
    - El único acceso posible al tipo debe ser a través de las operaciones públicas de éste